WEEK 4 · LESSON 21 OF 24 · HANDOUT

Human Oversight and Safe Failure

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=EUXevpbMFHU — open the lesson online to watch inside the course.

🎓 **Go deeper:** AI's Human Factor: Fei-Fei Li and Mira Murati — https://www.youtube.com/watch?v=9B02MzWwkSo

🎓 **Go deeper:** PyData London 2019: A practical guide towards algorithmic bias and explainability in machine learning — https://www.youtube.com/watch?v=vq8mDiDODhc

Your goal in this lesson: Design how people and models work together: a review band for uncertain cases, appeals, overrides, fallbacks when the model fails, and measures of whether oversight really works.

Humans in the loop — for real

"A human checks the decisions" often means a tired person clicking "approve" on 300 scores a day. Real oversight needs design:

- The model decides only where it is **confident**; uncertain cases go to people.
- People get **reasons** (Lesson 15), not just a number.
- People have time, training and the **authority to disagree**.
- Applicants can **appeal**, and appeals are tracked.
- When the model is down or wrong, there is a **fallback**.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

A review band

```
scores = model.predict_proba(test[FEATURES])[:, 1]

def route(score, low, high):
    if score >= high:
        return "approve"
    if score <= low:
        return "decline"                    # in our policy: decline means "refer with
reasons", never silent
    return "human review"

rows = []
for low, high in [(0.5, 0.5), (0.4, 0.6), (0.3, 0.7), (0.2, 0.8)]:
    routes = np.array([route(s, low, high) for s in scores])
    auto = routes != "human review"
    correct = (routes[auto] == "approve") == (test[TARGET].values[auto] == 1)
    rows.append({"band": f"{low}-{high}", "automated": f"{auto.mean():.0%}",
                 "accuracy on automated cases": f"{correct.mean():.1%}",
                 "cases for people per 1,000": int(round((~auto).mean() * 1000))})
print(pd.DataFrame(rows).to_string(index=False))
```

Wider bands mean the automated decisions are more accurate — but people must review more cases. Choose the band from your staff's real capacity, and give reviewers the hard cases with reasons attached.

Which group lands in the review band?

```
band = (scores > 0.3) & (scores < 0.7)
print(test.assign(in_band=band).groupby("area")["in_band"].mean().round(3).to_dict())
```

Here rural applicants land in the review band more often than urban ones (about 27% against 20%). If one group is sent to review much more often, they wait longer — another fairness question to watch.

Overrides teach you something

When reviewers disagree with the model, record it:

```
rng = np.random.default_rng(5)
reviewed = test[band].assign(score=scores[band])
reviewed["officer_decision"] = np.where(rng.random(len(reviewed)) < 0.5 + (reviewed["score"] -
0.5), "approve", "decline")
reviewed["model_suggestion"] = np.where(reviewed["score"] >= 0.5, "approve", "decline")
override = reviewed["officer_decision"] != reviewed["model_suggestion"]
print(f"override rate: {override.mean():.0%} of {len(reviewed)} reviewed cases")
for who in ["model_suggestion", "officer_decision"]:
    acc = ((reviewed[who] == "approve") == (reviewed[TARGET] == 1)).mean()
    print(f"accuracy of {who}: {acc:.1%}")
```

In this simulation the officers' decisions are **less** accurate than the model's suggestions on the reviewed cases — a sign they need better reasons or training. Track overrides and their outcomes. If officers are often right when they override, learn why — maybe the model misses information. If they are often wrong, they may need training or better reasons. An override rate near **0%** can mean "rubber-stamping": people no longer think.

Safe failure

```
def decide(applicant, model_available=True):
    try:
        if not model_available:
            raise RuntimeError("model service unreachable")
        p = float(model.predict_proba(pd.DataFrame([applicant])[FEATURES])[0, 1])
        return {"route": route(p, 0.3, 0.7), "source": "model", "score": round(p, 2)}
    except Exception as exc:
        # fallback: simple, written rules + a person; never a silent refusal
        risky = applicant["loan_rwf"] > 3 * applicant["income_rwf"] or
applicant["late_payments"] >= 3
        return {"route": "human review", "source": f"fallback ({exc})", "flag": "high risk" if
risky else "normal"}

a = {"income_rwf": 300_000, "loan_rwf": 250_000, "years_member": 4, "late_payments": 1,
"salaried": 1, "momo_tx_month": 35}
print(decide(a))
print(decide(a, model_available=False))
```

When the model fails, the service still works — with simple written rules and a person. Never let a technical failure turn into automatic refusals.

Appeals

Every applicant who is declined should receive: the main reasons, how to appeal, and the time limit for an answer. Track appeals: how many, how long they take, and how many change the decision — by group.

Key terms

- **Human oversight** — people able to understand, check and override AI decisions.
- **Review band** — scores where the model does not decide and a person does.
- **Override** — a person changes the model's suggestion.
- **Automation bias** — trusting a machine's answer too much.
- **Fallback** — what happens when the model is unavailable or unreliable.
- **Appeal** — a person's right to ask for a decision to be reviewed.

Remember this: Design oversight on purpose — a review band sized to real capacity, reasons for reviewers, tracked overrides and appeals, and a safe fallback that never refuses silently.

Name: _____ Date: _____

Activity

Task 1

With 20 officers reviewing 15 cases each per day and 1,000 applications per day, which band can you afford?

Task 2

Make the simulated officers better than the model and check what the numbers show.

Task 3

Add an "appeal" field to `decide` that tells declined applicants how to appeal.

Task 4

Write the fallback rules for your own capstone system.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a review band?
   - A. A server limit
   - B. A music group
   - C. A range of scores where a person, not the model, decides
   - D. A data column
2. What happens when the review band gets wider?
   - A. Automated decisions get less accurate
   - B. The model retrains
   - C. Nothing
   - D. Automated decisions get more accurate but people review more cases
3. What can an override rate near 0% suggest?
   - A. Rubber-stamping: people no longer really check
   - B. Too many officers
   - C. Perfect model
   - D. A network error
4. What should happen if the model service is down?
   - A. Shut the SACCO
   - B. Approve everyone
   - C. Fall back to simple written rules and a person
   - D. Refuse everyone
5. What should a declined applicant receive?
   - A. Only the score
   - B. An advert
   - C. Nothing
   - D. The main reasons, how to appeal and the time limit
6. What is automation bias?
   - A. A type of drift
   - B. A faster computer
   - C. Trusting a machine's answer too much
   - D. A fairness metric

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.